Trekking Management Application
Project Report

1) Student Details

Name: Sanskriti Raj
Roll Number: 23f1000417
Email: 23f1000417@ds.study.iitm.ac.in
About Me: I am a Diploma level student at BS in Data Science and Application at IIT Madras. I am currently learning App development, Deep learning and Quant finance. I am passionate to develop solutions of real world problems with the help of technologies.

---

2) Project Details

Project Title: Trekking Management Application

Problem Statement: To build a Web based application (**Trekking Management Application)** that allows **Admin, Trek Staff, and Users (Trekkers)** to interact with the system based on their roles.

Approach: This app is built using backend frameworks as Flask and I used a modular structure which is to segregate out the controllers, models, and database parts. It allows users to discover treks easily, get realtime trek information, get helped by personal trek guide throughout the journey, and can better plan their trek.

---

3) AI/LLM Declaration

- I used AI assistance for improving the working of backend routes.
- The extent of AI/LLM usage is 25-30%.
- Other application design, feature implementation, integration and testing were done manually.

---

4) Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend framework and routing by controllers.py
SQLAlchemy	ORM is defining models and query the SQLite database

Jinja2	Template engine used to render dynamic, role specific HTML pages
HTML/CSS and Bootstrap 5	Frontend styling, responsive layout, badges, tables and cards.
Flask-Login	For user authentication and session management
SQLite	Lightweight file based relational database i.e. trekking.db

---

5) Database Schema / ER Diagram

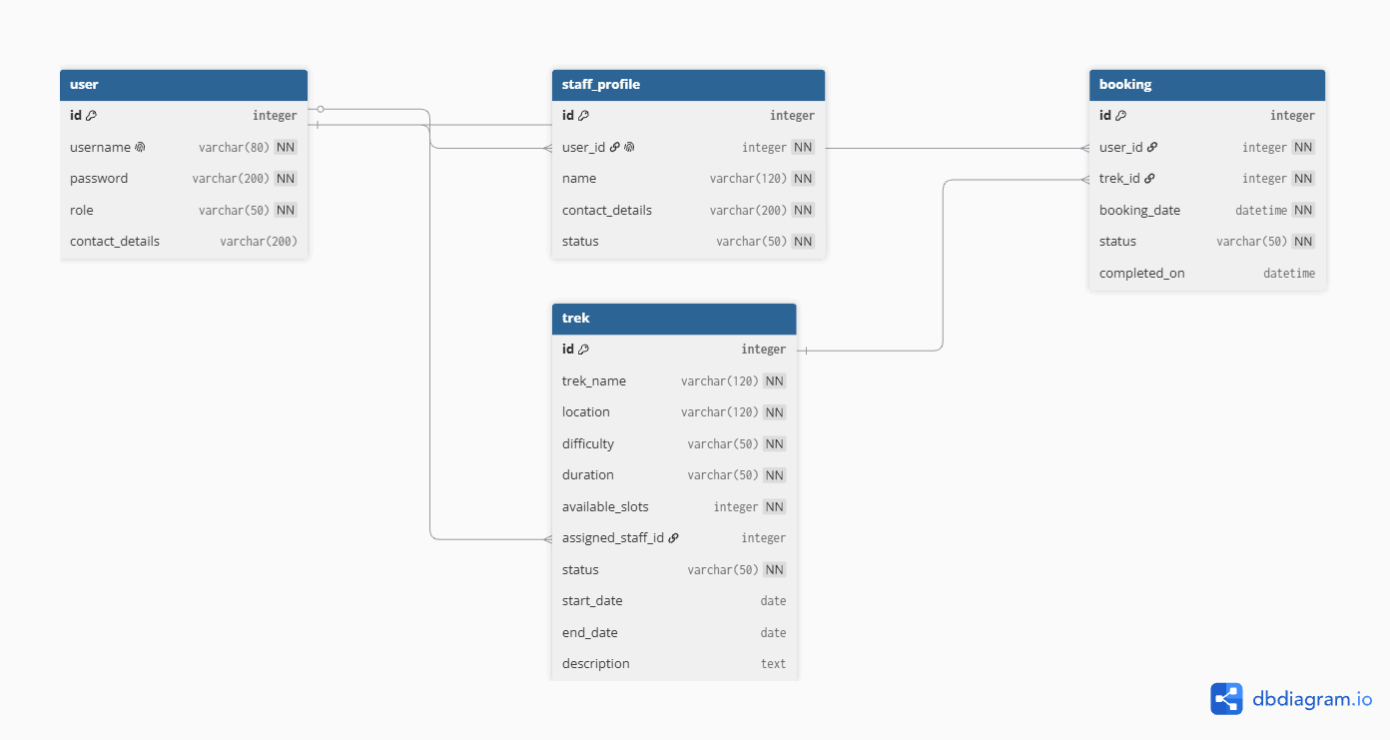
Tables:

1. **User** (id, username, password, role, contact_details) - role is one of Admin / Trek Staff / Trekker / Blocked.
2. **Staff Profile** (id, user_id → User, name, contact_details, status) - one-to-one with User; status is Pending / Approved / Blacklisted.
3. **Trek** (id, trek_name, location, difficulty, duration, available_slots, assigned_staff_id → User, status, start_date, end_date, description).
4. **Booking** (id, user_id → User, trek_id → Trek, booking_date, status, completed_on) - status is Booked / Cancelled / Completed.

Relationships:

- One-to-One : User (Trekker) ↔ Staff Profile
- One-to-Many : Trek Staff ↔ Trek
- One-to-Many : Trek ↔ Booking
- One-to-Many : User (Trekker) ↔ Booking

ER-Diagram:



---

6) API Resource Endpoints

Authentication

End Point	Method	Description
/register	GET/POST	Trekker/Trek Staff self-registration
/login	GET/POST	Authenticate user, create session
/logout	GET	Clear session and log out

Admin

/admin/dashboard	GET	Summary stats (treks, users, staff, bookings)
/admin/treks	GET/POST	List/search all treks
/admin/treks/add	POST	Create new trek
/admin/treks/edit/<id>	GET/POST	Edit trek details
/admin/treks/delete/<id>	POST	Delete a trek

/admin/staff	GET/POST	List staff, approve/blacklist
/admin/staff/add	POST	Add staff directly (auto-approved)
/admin/staff/delete/<id>	POST	Remove staff member
/admin/users	GET/POST	List/search trekkers, block/unblock
/admin/users/delete/<id>	POST	Remove trekker account
/admin/bookings	GET	View all system booking records

Trek Staff

/staff/dashboard	GET	Assigned treks + participant overview
/staff/profile/update	POST	Update staff profile
/staff/trek/manage/<id>	GET	View trek participants
/staff/trek/update-status/<id>	POST	Update trek status/slots
/staff/trek/<id>/participant/<bid>/cancel	POST	Remove a participant

Trekker

/trekker/dashboard	GET	Browse/search open treks
/trekker/profile/update	POST	Update trekker profile
/trekker/book/<id>	POST	Book a trek
/trekker/booking/cancel/<id>	POST	Cancel a booking
/trekker/history	GET	View completed trek history

---

7) <u>Architecture and Features</u>

Architecture Overview:

- **app.py :** It starts the application and sets up Flask and SQLAlchemy, and creates the default admin user.
- **application/database.py :** the shared SQLAlchemy db instance.
- **application/models.py :** User, StaffProfile, Trek and Booking models and their relationships.

- **application/controllers.py :** a single Blueprint holding all routes, grouped by role (auth, admin, staff, trekker), plus shared helpers, a before_request access guard and error handlers.
- **templates/ :** Jinja2 templates (base.html layout + one template per screen) extending a common Bootstrap-styled base with a role-aware navbar.
- **init_db.py :** one-time script to drop and recreate tables from the updated models, since db.create_all() does not alter existing tables.

Implemented Features:

- Role-based registration and login (Trekker can self-register and can log in, Trek Staff can self-register but can log in only after Admin approval, Admin account is default and can't register by self).
- Role-based dashboards and route-level access restriction using a before_request decorator for paths /admin, /staff and /trekker
- **Admin**: Dashboards with KPIs, creation, read, update or delete of treks with staff allocation, approval/blacklisting of staff. Search, blacklist or remove and full view of booking records can be done for users(trekkers).
- **Trek staff**: Profile update, treks assigned with number of participants, status or slot update for each trek, participant management
- **Trekker**: Profile update, trek search or filter by location and difficulty, booking with duplicate-booking and full or closed-trek prevention, booking cancellation, and have complete trek history.
- Data Validation: Users, staff, or treks with existing bookings or assignments cannot be deleted. Trek slot availability is updated automatically when a booking is done or cancelled. When a trek is marked as completed, all related active bookings are automatically marked as completed with the completion date.

---

8. Video Presentation

Drive Link: TMA-MAD1 Video Presentation